



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure - objective metric that scores how successfully an agent's actions modify the environment state
Environment - The external world state, contexts, and systems the agent inhabits
Actuators - physical,software mechanisms the agent triggers to execute actions
Sensors - hardware,API ports through which the agent receives its percept sequences

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The variables that represent the physical environment state include `self.gridsize`, `self.walls`, `self.food`, `self.agent_pos` and `self.opponents`. These define the layout and current physical objects present in the grid world.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

It is multiagent. The presence of `self.opponents` means there are other active entities operating in the same environment, potentially competing or interacting with the main agent.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

a percept sequence is the complete history of everything the agent has ever perceived through its sensors up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

It represents the Sensors.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

It is Partially Observable. The agent only receives local or specific information (like immediate surroundings or radar data) instead of the complete global state of all walls, traps and hidden elements across the entire grid at once.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance measure.
The objective metric that grades environmental success is being updated.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

AI performance must be measured by how successfully it achieves its external goals in the real world (task completion, safety) rather than how hard its internal CPU or code is working, because an agent could use a lot of compute power while failing the actual task.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`_init_`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

It makes the environment more partially observable (or keeps it partially observable) because the agent lacks the sensor data to detect the hidden traps, leading to incomplete information about the state of the world.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos)` in `self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

it provides critical local warning information. With this sensor, the agent can make informed decisions to avoid toxic traps, thereby reducing negative score penalties and maximizing its expected performance score overtime.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score`

by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic example is a vacuum cleaner agent that is rewarded for cleaning dirt but if the performance metric fails to penalize making a mess the agent learns to dump the dirt out and clean it up repeatedly to infinitely farm reward points without actually doing useful work.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING